Hamiltonian cycles after the ETH refutations

Notes for a mathematical reviewer — Vaibhav Tulsyan, 27 October 2026

1. Summary

Context. On 26 October 2026, OpenAI and Anthropic each posted a claimed refutation of the Exponential Time Hypothesis (ETH):

- both give deterministic algorithms that count the satisfying assignments of k-CNFs in time about 2^{O(n/√log n)};
- both come with Lean 4 proofs;
- both are still under expert review (Ryan Williams is organising a review group with Impagliazzo and Paturi).

Neither paper covers Hamiltonian cycle or other connectivity problems. Writing “the chosen edges form one cycle” as a SAT formula needs about n log n variables, which is too many for the new counters to help.

Results. All of them are for graphs with |E| = O(n), and all were obtained with heavy AI assistance (Claude Code).

	Result	Status
R1	Parity of #HC(G), deterministically, in subexponential time (Cut&Count encoding, mod 2). Randomised decision. Directed versions.	**Machine-checked in Lean** on top of OpenAI’s and Anthropic’s own Lean proofs, both rebuilt from source. Only the three standard axioms are used.
R2	**Exact #HC(G), deterministically.** Time 2^{δn} for every δ > 0 when |E| ≤ Dn. A single algorithm runs in 2^{O(max(m,n)/√log n)} for all graphs. Corollaries: deterministic decision, search, and Hamiltonian paths.	Written proof (Section 2 onward of this document). Independent AI referee: sound, conditional on [A]. Exact numerical checks pass. Identity (3.1) is in Lean. Full Lean formalisation is in progress.
R3	The same template gives deterministic 2^{δn} algorithms for directed HC, TSP (polynomial weights), Steiner tree, FVS, all forests (Tutte T(x,1)), longest path, k-leaf spanning trees, and others.	Conditional on R2. Each identity is checked numerically. Not refereed.

What R2 depends on. It relies on three results of Anthropic’s paper [A]:

- Theorem A(b), the clique algorithm with table queries;
- Lemma 8, which reduces networks to cliques;
- Theorem B, for the √log n rate.

All three are proved in Anthropic's Lean, but there they are applied only to networks that come from CNFs. We apply them to a different network, so that application is a paper argument.

2. The argument of R2 in one page

1. **Spanning trees detect Hamiltonicity.** A 2-factor X is either one n-cycle, with exactly n spanning trees, or a disjoint union of cycles, with none. Hence n·#HC(G) = Σ_{X 2-factor} τ(V, X).
2. **Fermionic matrix-tree theorem** (Caracciolo–Jacobsen–Saleur–Sokal–Sportiello 2004). τ(V,X) is the top Grassmann coefficient of Π_{e=ij∈X} (1 + (ψ̄_i − ψ̄_j)(ψ_i − ψ_j)), with ψ_r = ψ̄_r = 0. The write-up proves this directly: Berezin Cauchy–Binet plus det(reduced incidence matrix)² = [spanning tree].
3. **Blobs.** Partition V into g blobs. Each crossing edge expands into 5 labelled pieces, so it costs ⌈$\log_2 5$⌉-ish bits. Regrouping the generators blob by blob gives a Koszul sign. The sign factorises as (per-blob local signs) × (−1)^{Σ_{a<c<b<d} P_ab P_cd}, a sum over interleaving chords, where P_ab is the parity of the odd pieces on the blob pair (a,b).
4. **Network.** Charge each interleaving pair to one blob using parity-copy bits, which costs O(g^3) bits. This gives a network in the sense of [A, Def. 2]:
   - arbitrary integer node tables;
   - equality-test edges;
   - about 4.64|E| + O(g^3) edge bits.

   High-degree vertices are handled by branching on their two cycle edges. Then [A]'s algorithm evaluates the network.

3. Where a careful reader is most useful

1. **The sign rule.** Proposition 4.2 (sign rule) and Proposition 4.4 (sign factorisation with the interleaving term), together with Lemma 4.3 (c_kl mod 2 for disjoint chords). This is the delicate step. The numerical ablations show that dropping the interleaving term gives wrong values.
2. **Fit with [A].** Does the network in Section 5 meet [A]'s Definition 2 and the hypotheses of Lemma 8 / Theorem A(b)? In particular check the per-node bit bound P, which forces the branching of Lemma 6.1, and the bookkeeping in Sections 6.3–6.4.
3. **Novelty.** Is the combination known? We believe identity 1 is folklore, and the fermionic matrix-tree theorem is classical. The algorithm could not have existed before [A].
4. **Mathematical correctness of [A] itself.** Beyond its Lean statement, this is outside our scope but relevant.

4. Verification done so far

- **Lean.**

    - R1 is complete, on top of both labs' proofs.
    - We rebuilt OpenAI's proof (11,492 jobs) and Anthropic's (9,768 jobs). Both rest only on propext, Classical.choice and Quot.sound, and a fresh `leanchecker` replay of OpenAI's passes.
    - For R2, the identity n·#HC = Σ_X τ(X) is proved (`ExactHC.lean`). The other modules are being formalised; the plan is in `research/lean-blueprint.md`.
- **Numerics.** These use exact integer arithmetic and independently written code.
    - The fresh referee code ran about 1,100 tests on random graphs and random partitions into 2–6 blobs, with random roots and random integer weights. There were 0 failures.
    - Every intermediate identity of the write-up is also checked (`proof_checks.py`).
- **Referee.** An independent AI referee derived the sign rule itself before reading the write-up. Verdict: sound, conditional on [A]. Report: `research/review-exact-count.md`.

5. Pointers

- **Repository:** https://github.com/xennygrimmato/eth-hamiltonicity
    - `research/exact-hc-theorem.md`: this write-up.
    - `research/review-exact-count.md`: the referee report.
    - `lean/`: the Lean proofs.
    - `research/scripts/`: the checks.
- **[A]** Claude (Anthropic), *A 2^{o(n)} Time Algorithm for 3-SAT*, 26 Oct 2026, github.com/anthropics/formal-math (folder `eth`).
- **[O]** OpenAI, *The Exponential Time Hypothesis is False*, 26 Oct 2026, github.com/openai/math.
- Cygan, Nederlof, Pilipczuk, Pilipczuk, van Rooij, Wojtaszczyk, *Solving connectivity problems parameterized by treewidth in single exponential time*, FOCS 2011 (Cut&Count).
- Caracciolo, Jacobsen, Saleur, Sokal, Sportiello, *Fermionic field theory for trees and forests*, PRL 93 (2004).

The full write-up of R2 follows.

Exact counting of Hamiltonian cycles in sparse graphs in time 2^{δn}

27 Oct 2026. This is the referee-ready write-up of the Track C construction (`exact-count-lowrank.md`). The construction is re-derived here from scratch, with full proofs. Conventions are fixed explicitly, and the parameter bookkeeping is done against the exact hypotheses of the imported results. The algorithmic theorems are **conditional**: they import a clique-evaluation algorithm and a network-to-clique reduction from Anthropic's manuscript *A 2^{o(n)} time algorithm for 3-SAT* (cited as [A]). §9 says exactly what is imported. Everything else is proved here.

Verification scripts, all in exact integer arithmetic: `research/scripts/proof_lib.py`, `proof_checks.py` and `proof_params.py`. §8 gives the details.

---

0. Results

Graphs are finite, simple and undirected. Throughout, G = (V, E), n = |V| and m = |E|. #HC(G) is the number of Hamiltonian cycles of G, each counted once as a subgraph (so #HC(K_4) = 3), and #HC(G) = 0 for n ≤ 2. Time is counted as in [A, §3]: operations on words long enough to hold an address, plus table queries.

Theorem 1. For every real δ > 0 and every D ≥ 1 there is a deterministic algorithm that, given a graph G with n vertices and at most D·n edges, outputs #HC(G) within O(2^{δn}) operations.

Theorem 2. There are an absolute constant C* and one deterministic algorithm that, given any graph G with n ≥ 2 vertices and m edges, outputs #HC(G) within 2^{C*·max(m,n)/√(log n)} operations. In particular it runs in time 2^{O(D·n/√log n)} on graphs with m ≤ D·n, where the O-constant is absolute and D may depend on n.

Corollary 3 (decision, search and paths). Within the same time bounds, multiplied by m + 1, one can also do the following, deterministically:

- decide Hamiltonicity;
- output a Hamiltonian cycle if one exists;
- count, decide and find Hamiltonian paths. For paths, apply the bounds to m + n edges and n + 1 vertices.

Conditionality. Theorems 1 and 2 and Corollary 3 assume three results of [A]:

- Lemma 8, case (i): the reduction from general networks to cliques;
- Theorem A(a),(b): one algorithm for the (K_ℓ, λ)-cliques of all levels ℓ;
- the existence of pieces (Definition 7): Theorem B, with ρ = 1/√2, for Theorem 2; for Theorem 1, Lemma 2's pieces with ρ = 11/12 also suffice.

For Theorem 1 the main-text pair Lemma 1 + Proposition 3 of [A] can replace all of these. Nothing else from [A] is used, in particular not its CNF Main Theorem. §9 lists the Lean counterparts of the imported statements. The statements are recalled verbatim in §1.

Gaps found while writing this up. The original note has no gap in its mathematics. Three points needed care, and are handled below:

1. Definition 2 bounds the *largest* node (P), not just the total. With unbounded maximum degree, one blob may have to carry Θ(n) crossing-edge bits, and then 2gP/K is not small. This is fixed by branching on the two cycle edges at each vertex of degree > n/g (Lemma 6.1, at most n^{4Dg} branches), followed by a degree-balanced partition (Lemma 6.2).
2. The interleaving sign costs $O(g^3)$ bits in total, not $O(g^4)$, and at most g^2 bits per node (Proposition 5.4).
3. For the sharper bound, $g = 3K^8$ must stay at most n^{8/30}, so that the g^3 copy bits and the n^{O(Dg)} branching are $o(n/\sqrt{\log n})$. Then $K \geq n^{1/300}$, which still gives $e_\ell = O(1/\sqrt{\log n})$ (§6.4).

---

1. Imported definitions and results (from [A], quoted)

Conventions of [A]: log = $\log_2$, [x = u] ∈ {0,1}, [K] = {1,...,K}.

Definition 1 [A]. $F : X \times X' \to Z$ *factors through c bits* if there are $f : X \times \{0,1\}^c \to Z$ and $f' : X' \times \{0,1\}^c \to Z$ with $F(z,z') = \Sigma_{\theta \in \{0,1\}^c} f(z,\theta)\, f'(z',\theta)$. Observation (b) of [A]: for any $\pi : X \to \{0,1\}^c$ and $\pi' : X' \to \{0,1\}^c$, $[\pi(z) = \pi'(z')] = \Sigma_\theta\, [\pi(z)=\theta][\pi'(z')=\theta]$ factors through c bits.

Definition 2 [A]. Let P, B, τ ≥ 1 be integers. A *network on J with parameters P, B, τ* has:

- for each node a ∈ J, a finite set S_a of settings, encoded as bit strings of one fixed length $P_a \leq P$, and a function $W_a : S_a \to Z$;
- for each unordered pair {a,b}, a function $\Psi_{a,b} : S_a \times S_b \to Z$ (symmetric), an integer $c_{a,b} = c_{b,a} \geq 0$, and a factorization $(f_{a,b}, f_{b,a})$ of $\Psi_{a,b}$ through $c_{a,b}$ bits.

The conditions are:

- i. $\Sigma_{b \neq a}\, c_{a,b} \leq P$ for every a;
- ii. all values of all W_a, $\Psi_{a,b}$, $f_{a,b}$ are below 2^B in absolute value;
- iii. an algorithm receives P, B, P_a and $c_{a,b}$, and decides membership in S_a or computes any value of W_a, $\Psi_{a,b}$, $f_{a,b}$ within τ operations.

The value is $Z = \Sigma_{z \in \Pi S_a}\, \Pi_a\, W_a(z_a)\, \Pi_{\{a,b\}}\, \Psi_{a,b}(z_a,z_b)$. The total is $U = \Sigma_{(a,b):\, a \neq b}\, c_{a,b}$, which counts every shared bit from both of its ends.

Definition 3 [A]. In the (K,λ)-clique with tables $W_1,\ldots,W_K$, a setting of node w is a tuple $(z_w(w'))_{w' \neq w}$ *of λ-bit strings, and* $\Psi\{w,w'\} = [z_w(w') = z_{w'}(w)]$. The tables are queried at unit cost.

Lemma 8 [A, App. A], case (i). Let K ≥ 2, $g = 3K^8$, $\zeta = 1/(2K)$. Take a network on [g] with parameters P, B, τ and total U. Put

$$L = \lceil (U + gP/K)/K^2 \rceil, \qquad B' = \lceil 2gP/K \rceil + g^2 B.$$

Suppose an algorithm outputs the value of every (K,L)-clique with |W_w| < 2^{B′} within T operations. Then the value of the network is computed within

```
(K^g + T · 2^{2gP/K} · τ) · (g(P+B))^{C₁}
(1.1)
```

operations, by one procedure, for an absolute constant C_1. The printed statement reads $K^g + T\cdot 2^{\{2gP/K\}\cdot\tau\cdot(g(P+B))}\{C_1\}$. (1.1) is the form proved in Lean as `ExplicitCost.groupingExplicit`, and it is weaker by at most a polynomial factor, which is irrelevant below.

Theorem A [A, App. A]. Suppose there are pieces with parameters (ρ, c) (Definition 7 of [A]), and let $\alpha' = \log(1/\rho)$. Fix an integer $x_0 \geq 18$ with $2^{x_0} \geq c + 5$, and define K_ℓ, e_ℓ by (40) of [A]: $K_0 = 2^{x_0}$, $K_{\ell+1} = g_\ell^2$, $g_\ell = 2^{3\ell+x_0} K_\ell$, $e_0 = ½$, $e_{\ell+1} = (\rho + 3\zeta_\ell)e_\ell + 3/K_\ell$.

- a. $\log K_\ell = (3x_0+6)2^\ell - 6\ell - 2x_0 - 6$, so $K_\ell < K_{\ell+1} \leq K_\ell^{10}$; and $½\rho^\ell \leq e_\ell \leq 0.85\,\rho^\ell$, where $\rho^\ell \leq ((3x_0+6)/\log K_\ell)^{\alpha'}$.
- b. One deterministic algorithm, given $\ell \geq 0$, λ, $B \geq 1$ and query access to tables with $|W_w| < 2^B$, outputs the value of the (K_ℓ, λ)-clique within

  ```
  2^{e_ℓ K_ℓ² λ} · (K_ℓ(λ+B))^{C₀ K_ℓ²}
  (1.2)
  ```

  operations, with C_0 depending only on c.

Theorem B [A]. There are pieces with parameters $(1/\sqrt{2}, c)$ for an absolute constant c. Lemma 2 of [A] gives pieces with parameters $(11/12, c)$ for some $c \geq 8$.

Lemma 1 / Proposition 3 [A] (alternative route for Theorem 1). If (K,e) is achievable and $g = 3K^8$, every network on [g] is evaluated within $C_1\cdot 2^{\{e(U+gP/K)+2gP/K\}\cdot(P+B)}\{C_1\}\cdot\tau$ operations, with $C_1 = C_1(K,e)$. For every $\eta > 0$, (K,η) is achievable for infinitely many K.

---

2. Grassmann algebra: the facts used

Let Γ be a finite set of *generators* with a total order ≺ (any order). The exterior algebra Λ(Γ) over Z is the free Z-module with basis $\{e_S : S \subseteq \Gamma\}$ and product

```
e_S · e_T = 0 if S ∩ T ≠ ∅,        e_S · e_T = (−1)^{#{(s,t) ∈ S×T : t ≺ s}} e_{S∪T}
otherwise.
```

This is the exterior algebra of Z^Γ (Bourbaki, *Algebra* III §7). It is associative with unit $e_\varnothing$. Writing γ for $e_{\{\gamma\}}$, we have $\gamma\gamma' = -\gamma'\gamma$ and $\gamma^2 = 0$, and e_S is the product of the elements of S in increasing ≺-order. A product of generators therefore does not depend on ≺. All statements below are about products and sums of generators, so they do not depend on ≺: changing ≺ only re-signs the basis. (The scripts randomize ≺ on purpose.)

Parity. $\Lambda = \Lambda^0 \oplus \Lambda^1$ with Λ^p spanned by the e_S with $|S| \equiv p \pmod 2$. An element of Λ^0 or Λ^1 is *homogeneous*, and 0 is homogeneous of both parities. Λ^0 is a subring.

Fact 2.1 (graded commutativity). For x ∈ Λ^p and y ∈ Λ^q, xy = (−1)^{pq} yx. In particular Λ^0 is central.

Proof. For disjoint S, T, every pair in S × T is counted in exactly one of the two exponents #{t ≺ s} and #{s ≺ t}, so e_S e_T = (−1)^{|S||T|} e_T e_S. Extend bilinearly. ∎

Lemma 2.2 (Koszul sign rule). Let y_1,…,y_M be homogeneous of parities p_1,…,p_M, and let σ be a permutation of [M]. Then

```
y_{σ(1)} y_{σ(2)} ⋯ y_{σ(M)} = (−1)^{N(σ)} y_1 y_2 ⋯ y_M,
N(σ) = Σ_{s<t, σ⁻¹(s) > σ⁻¹(t)} p_s p_t.
```

The sum runs over the pairs whose relative order σ reverses.

Proof. Induct on the number of reversed pairs. If σ ≠ id, some adjacent positions j, j+1 have σ(j) > σ(j+1). Exchanging these two factors multiplies the product by (−1)^{p_{σ(j)} p_{σ(j+1)}} (Fact 2.1). It removes exactly the pair {σ(j), σ(j+1)} from the set of reversed pairs and changes no other relative order. ∎

Sub-algebras and top coefficients. For Γ′ ⊆ Γ, Λ(Γ′) ⊆ Λ(Γ) is the span of the e_S with S ⊆ Γ′. Let |Γ′| be even and ω = ±e_{Γ′}. For x ∈ Λ(Γ′), [ω](x) denotes the coefficient c with x − cω ∈ span{e_S : S ⊊ Γ′}.

Lemma 2.3 (block factorization of the top coefficient). Let Γ = Γ_1 ⊔ ⋯ ⊔ Γ_g with every |Γ_a| even (blocks may be empty). Let ω_a = ±e_{Γ_a}, Ω = ω_1 ω_2 ⋯ ω_g, and x_a ∈ Λ(Γ_a). Then

```
[Ω](x_1 x_2 ⋯ x_g) = Π_a [ω_a](x_a).
```

Proof. Ω = ±e_Γ, so [Ω] is defined. Write x_a = [ω_a](x_a)·ω_a + r_a, where r_a is a combination of e_S with S ⊊ Γ_a. Expand the product. The term with no r_a is Π_a[ω_a](x_a)·Ω. Every other term is a combination of products e_{S_1}⋯e_{S_g} with S_b ⊆ Γ_b and S_a ⊊ Γ_a for some a. Such a product is 0 or ±e_{∪S_b}, with ∪S_b ≠ Γ, so it does not contribute to [Ω]. ∎

---

3. Hamiltonian cycles through spanning trees

Fix G = (V,E) with n ≥ 3 and a root r ∈ V. A *2-factor* is X ⊆ E such that every vertex has degree exactly 2 in (V,X). τ(V,X) denotes the number of spanning trees of the graph (V,X).

Lemma 3.1. n·#HC(G) = Σ_{X 2-factor of G} τ(V,X).

Proof. A 2-factor X makes (V,X) a disjoint union of cycles covering V. If (V,X) is connected it is a single n-cycle, that is, a Hamiltonian cycle. Its spanning trees are obtained by deleting one of its n edges, so τ = n. If (V,X) is disconnected, τ = 0. Conversely, the edge set of every Hamiltonian cycle is a connected 2-factor, and distinct Hamiltonian cycles have distinct edge sets. ∎

Generators. Γ = {$\bar\psi$_v, ψ_v : v ∈ V\{r}}, in any order. We put $\bar\psi$_r = ψ_r := 0 (the zero element), ω_v := $\bar\psi$_v ψ_v ∈ Λ^0, and for S ⊆ V

```
Ω_S := Π_{v ∈ S\{r}} ω_v ,        q_e := (ψ̄_i − ψ̄_j)(ψ_i − ψ_j) ∈ Λ⁰    for e = {i,j}.
```

The order of the factors of Ω_S does not matter, because they are even and central. Ω_S = ±e_{Γ_S}, where Γ_S is the set of generators of the vertices of S. q_e does not depend on the orientation of e, since both factors change sign.

Lemma 3.2 (Berezin form of Cauchy–Binet). Let N = n − 1 and b_1,...,b_N ∈ Z^{V\{r}}. Put θ(b) = Σ_u b(u)ψ̄_u and θ(b) = Σ_u b(u)ψ_u. Then

```
[Ω_V]( θ̄(b_1)θ(b_1) · θ̄(b_2)θ(b_2) ⋯ θ̄(b_N)θ(b_N) ) = det(B)²,
```

where B is the N×N matrix with columns b_k. Its rows are taken in any fixed order of V\{r}, and det(B)² does not depend on that order.

Proof. Enumerate V\{r} as u_1,...,u_N.

1. Lemma 2.2: moving each θ(b_k) to the left past θ(b_1),...,θ(b_{k−1}) gives Π_k θ(b_k)θ(b_k) = (−1)^{N(N−1)/2} θ(b_1)⋯θ(b_N) · θ(b_1)⋯θ(b_N).
2. Expanding multilinearly, θ(b_1)⋯θ(b_N) = Σ_{φ:[N]→[N]} Π_k b_k(u_{φ(k)}) ψ̄*{u{φ(1)}}⋯ψ̄*{u{φ(N)}}. Only bijections φ survive, because ψ̄² = 0. For those, ψ̄*{u{φ(1)}}⋯ψ̄*{u{φ(N)}} = sgn(φ) ψ̄*{u_1}⋯ψ̄*{u_N} by Lemma 2.2. Hence θ(b_1)⋯θ(b_N) = det(B)·ψ̄*{u_1}⋯ψ̄*{u_N}, and likewise θ(b_1)⋯θ(b_N) = det(B)·ψ_{u_1}⋯ψ_{u_N}.
3. Undoing step 1 for the standard vectors gives ψ̄*{u_1}⋯ψ̄*{u_N} ψ_{u_1}⋯ψ_{u_N} = (−1)^{N(N−1)/2} ω_{u_1}⋯ω_{u_N} = (−1)^{N(N−1)/2} Ω_V.
4. The two signs cancel, and the product is det(B)²·Ω_V. ∎

Lemma 3.3 (fermionic matrix-tree theorem). For every X ⊆ E, [Ω_V] Π_{e∈X}(1 + q_e) = τ(V,X).

Proof. The factors are even and commute, so Π_{e∈X}(1+q_e) = Σ_{F⊆X} Π_{e∈F} q_e. The term for F lies in the span of the e_S with |S| = 2|F|, while Ω_V = ±e_Γ with |Γ| = 2N. So only |F| = N contributes. For e = {i,j}, q_e = θ(b_e)θ(b_e) with b_e = 1_i − 1_j restricted to V\{r} (that is, 1_i if j = r). By Lemma 3.2, F contributes det(B_F)², where B_F is the reduced incidence matrix of F.

- If F (with |F| = n − 1) is not a spanning tree, then (V,F) is disconnected. A connected graph on n vertices with n − 1 edges is a tree. Let C be a component with r ∉ C. The rows u ∈ C of B_F sum to 0: a column of an edge inside C has +1 and −1 there, and other columns vanish on C. So det B_F = 0.
- If F is a spanning tree, then det B_F = ±1, by induction on n. For n = 1 the matrix is empty. For n ≥ 2, pick a leaf v ≠ r (a tree has ≥ 2 leaves). Row v has one nonzero entry, ±1, in the column of its edge f. Expanding along it gives det B_F = ±det B_{F−f}, the reduced incidence matrix of the spanning tree F − f of V − v.

Hence the coefficient is #{spanning trees F ⊆ X} = τ(V,X). ∎

(By Cauchy–Binet this also gives τ(V,X) = det L_X^{(r)}, Kirchhoff's theorem. That is not needed, but it is checked numerically, as C2 in §8.)

Corollary 3.4.

```
n·#HC(G) = Σ_{x∈{0,1}^E} Π_{v∈V}[deg_x(v) = 2] · [Ω_V] Π_{e: x_e=1}(1 + q_e).
```

---

4. Blobs, pieces and the sign rule

Conventions (fixed once and for all).

- *Partition.* $\beta : V \to [g]$ is an arbitrary map, and the blobs are $V_a = \beta^{-1}(a)$. Blobs may be empty, and r may lie in any blob. The blob indices $1 < 2 < \cdots < g$ are the "line" on which chords interleave.
- *Edges.* An edge is *internal* to a if both ends lie in V_a; the set of these is $E_{int}(a)$. The other edges are *crossing*.
- *Crossing order.* The crossing edges are listed in an arbitrary but fixed total order $e_1,\ldots,e_m$. Write $k < l$ for e_k before e_l.
- *Ends.* A crossing edge e_k has a *low end* s_k and a *high end* t_k, with $a_k := \beta(s_k) < b_k := \beta(t_k)$. Its *chord* is $\{a_k, b_k\}$.
- *Incidences.* $\delta(a) := \{k : a \in \{a_k, b_k\}\}$ is the set of crossing edges at a. For $a < b$, $E(a,b) := \{k : (a_k, b_k) = (a,b)\}$, $e_{ab} := |E(a,b)|$, and $e_{ba} := e_{ab}$.
- *Algebras.* Γ_a is the set of generators of $V_a \setminus \{r\}$, $\Lambda_a := \Lambda(\Gamma_a)$ and $\omega_a := \Omega_{V_a}$. Then $\Omega_V = \omega_1 \cdots \omega_g$, and Lemma 2.3 applies to the partition $\Gamma = \sqcup_a \Gamma_a$.

Labels and pieces. The label set is $L = \{N, 0, 1, 2, 3\}$, and $\pi(\ell) := [\ell \in \{2,3\}]$ is the *parity* of ℓ. For the crossing edge e_k (ends $s = s_k$, $t = t_k$):

ℓ	$\alpha_\ell(k) \in \Lambda_{a_k}$	$\beta_\ell(k) \in \Lambda_{b_k}$	parity $\pi(\ell)$
N	1	1	0
0	1	$1 + \bar\psi_t \psi_t$	0
1	$\bar\psi_s \psi_s$	1	0
2	$\bar\psi_s$	$-\psi_t$	1
3	ψ_s	$\bar\psi_t$	1

Label N means "$e_k \notin X$", and labels 0–3 mean "$e_k \in X$" together with one term of $1 + q_{e_k}$.

Lemma 4.1.

- a. $\alpha_\ell(k)$ and $\beta_\ell(k)$ are homogeneous of parity $\pi(\ell)$. This includes the cases $s = r$ or $t = r$, where some pieces are 0 or reduce to 1.
- b. $\alpha_N \beta_N = 1$ and $\Sigma_{\ell\in\{0,1,2,3\}} \alpha_\ell(k)\, \beta_\ell(k) = 1 + q_{e_k}$.

Proof. (a) Read off the table. (b) By bilinearity, $q_e = \bar\psi_s\psi_s - \bar\psi_s\psi_t - \bar\psi_t\psi_s + \bar\psi_t\psi_t$ for any elements, zero included, and $-\bar\psi_t\psi_s = \psi_s\bar\psi_t$. The four products in the table are $1 + \bar\psi_t\psi_t$, $\bar\psi_s\psi_s$, $-\bar\psi_s\psi_t$ and $\psi_s\bar\psi_t$, which sum to $1 + q_e$. ∎

For a labelling $\ell \in L^m$, write $\pi_k := \pi(\ell_k)$. Define the piece of k at a, $\gamma_{k,a}(\ell) := \alpha_{\ell_k}(k)$ if $a = a_k$ and $\beta_{\ell_k}(k)$ if $a = b_k$, and the ordered product of a's pieces:

```
Γ_a(ℓ) := Π→_{k∈δ(a)} γ_{k,a}(ℓ)      (factors in increasing crossing order k) ∈ Λ_a,
```

```
c_{kl} := #{(x,y) ∈ {a_k,b_k} × {a_l,b_l} : y < x},       ε(ℓ) := (−1)^{Σ_{k<l} π_k π_l
c_{kl}}.
```

Proposition 4.2 (sign rule). For every partition, every crossing order and every ℓ ∈ L^m:

```
Π_{k=1}^{m} α_{ℓ_k}(k) β_{ℓ_k}(k) = ε(ℓ) · Γ_1(ℓ) Γ_2(ℓ) ⋯ Γ_g(ℓ).
```

Proof. The left side is the product of the sequence Y = (α(1), β(1), α(2), β(2), …, α(m), β(m)). Give the piece α(k) the key (a_k, k) and the piece β(k) the key (b_k, k). Sorting Y by key, lexicographically, lists blob 1's pieces in increasing k, then blob 2's, and so on. So its product is Γ_1⋯Γ_g. By Lemma 4.1(a) all pieces are homogeneous, so Lemma 2.2 applies, and the sign counts the pairs of pieces whose relative order is reversed:

- *Two pieces of the same k.* In Y, α(k) comes first. Its key is also smaller, since a_k < b_k. Not reversed.
- *A piece x of k and a piece y of l, with k < l.* In Y, x comes first. The keys are (blob(x), k) and (blob(y), l). Since k < l, the pair is reversed iff blob(y) < blob(x). The parity product is π_kπ_l.

So Γ_1⋯Γ_g = (−1)^{Σ_{k<l} π_kπ_l c_{kl}} · Π_k α(k)β(k). Multiply both sides by ε(ℓ) = ±1. ∎

Lemma 4.3 (c_kl for disjoint chords). If {a_k,b_k} ∩ {a_l,b_l} = ∅, then c_{kl} ≡ I(k,l) (mod 2), where I(k,l) := [a_k < a_l < b_k < b_l or a_l < a_k < b_l < b_k] is the indicator that the chords interleave. In particular c_{kl} mod 2 is symmetric and independent of the crossing order.

Proof. Put Y = {a_l, b_l}, whose elements differ from a_k and b_k. Then c_{kl} = #{y∈Y: y<a_k} + #{y∈Y: y<b_k} = 2#{y∈Y: y<a_k} + #{y∈Y: a_k<y<b_k}. The last number is odd iff exactly one endpoint of chord l lies strictly between a_k and b_k, which is iff the chords interleave. ∎

Local and interleaving parts. Two crossing edges k < l *share* a blob if their chords meet. For such a pair put s(k,l) := min({a_k,b_k} ∩ {a_l,b_l}); then k, l ∈ δ(s(k,l)). Define

```
σ_a(ℓ) := (−1)^{ Σ_{k<l: chords meet, s(k,l)=a} π_k π_l c_{kl} }        (a function of ℓ|
δ(a)),
P_{ab}(ℓ) := Σ_{k∈E(a,b)} π_k  mod 2        for a < b                     (a function of
ℓ|E(a,b)).
```

Proposition 4.4 (sign factorization).

```
ε(ℓ) = Π_{a=1}^{g} σ_a(ℓ) · (−1)^{ Σ_{a<c<b<d} P_{ab}(ℓ) P_{cd}(ℓ) }.
```

Proof. Split Σ_{k<l} π_kπ_l c_{kl} into the pairs whose chords meet and the pairs whose chords are disjoint.

- *Meeting chords.* Each such pair has exactly one s(k,l), so these pairs give Π_a σ_a.
- *Disjoint chords.* By Lemma 4.3 these pairs give, mod 2, Σ_{k<l: I(k,l)=1} π_kπ_l. Take an unordered pair {k,l} with disjoint interleaving chords, and let {a,b} (a < b) be the chord with the smaller minimum and {c,d} the other. Interleaving means a < c < b < d. Conversely, every quadruple a < c < b < d with k ∈ E(a,b) and l ∈ E(c,d) gives such

a pair. This is a bijection, so the sum equals Σ_{a<c<b<d} (Σ_{k∈E(a,b)} π_k) (Σ_{l∈E(c,d)} π_l) ≡ Σ_{a<c<b<d} P_{ab}P_{cd} (mod 2). ∎

Parity copies. Let Q_a := {(c,d) : a < c < d ≤ g}, so |Q_a| = C(g−a, 2). For y ∈ {0,1}^{Q_a} put

```
χ_a(ℓ, y) := (−1)^{ Σ_{(c,d)∈Q_a} y_{cd} · Σ_{b: c<b<d} P_{ab}(ℓ) }     (a function of ℓ|
δ(a) and y).
```

Lemma 4.5. If y^{(a)}*{cd} = P*{cd}(ℓ) for all a and all (c,d) ∈ Q_a, then Π_a χ_a(ℓ, y^{(a)}) = (−1)^{Σ_{a<c<b<d} P_{ab}P_{cd}}.

Proof. Σ_a Σ_{a<c<d} P_{cd} Σ_{c<b<d} P_{ab} = Σ_{a<c<b<d} P_{ab}P_{cd}. ∎

So every interleaving pair of chords {a,b}, {c,d} with a < c < b < d is charged to node a, which knows P_{ab}. Node a receives a copy of P_{cd} from node c, over the pair {a,c}.

Blob sums. For v ∈ V_a let deg_ℓ(v) := #{k ∈ δ(a) : v ∈ {s_k,t_k}, ℓ_k ≠ N}. Define

```
I_a(ℓ) := Σ_{x∈{0,1}^{E_int(a)}} Π_{v∈V_a}[deg_x(v) + deg_ℓ(v) = 2] · [ω_a]( F_a(x)
Γ_a(ℓ) ),
F_a(x) := Π_{e∈E_int(a): x_e=1} (1 + q_e) ∈ Λ_a ∩ Λ⁰.
```

I_a depends only on ℓ|δ(a). An empty blob has I_a = 1, and so does a blob consisting of r alone with no edges.

Theorem 4.6 (blob factorization). For every partition and every crossing order,

```
n·#HC(G) = Σ_{ℓ∈L^m} ε(ℓ) Π_a I_a(ℓ) = Σ_{ℓ∈L^m} Π_a σ_a(ℓ) χ_a(ℓ, P(ℓ)) I_a(ℓ),
```

where P(ℓ) means y^{(a)}*{cd} = P*{cd}(ℓ).

Proof. The second equality is Proposition 4.4 with Lemma 4.5. For the first:

1. Expand Π_a I_a. It becomes a sum over x_int ∈ {0,1}^{∪_a E_int(a)} of the degree indicators times Π_a [ω_a](F_a Γ_a).
2. By Lemma 2.3 (F_aΓ_a ∈ Λ_a), Π_a [ω_a](F_a Γ_a) = [Ω_V](Π_a F_aΓ_a).
3. The F_a are even and central, so Π_a (F_aΓ_a) = (Π_a F_a)·Γ_1⋯Γ_g.
4. By Proposition 4.2, ε(ℓ)Γ_1⋯Γ_g = Π_k α_{ℓ_k}(k)β_{ℓ_k}(k).
5. Fix x_int and the support T := {k : ℓ_k ≠ N}. The degree indicators say exactly that x := x_int ∪ 1_T is a 2-factor, and they do not depend on the labels in T.
6. By distributivity and Lemma 4.1(b), Σ_{ℓ: supp ℓ = T} Π_k α(k)β(k) = Π_{k∈T} (1+q_{e_k}).
7. Hence Σ_ℓ ε Π_a I_a = Σ_{x∈{0,1}^E} [x 2-factor] [Ω_V] Π_{x_e=1}(1+q_e) = n·#HC(G), by Corollary 3.4. ∎

---

5. The network

Fix G, r, β and a crossing order as in §4. κ : L → {0,…,4} is the encoding κ(N) = 0, κ(j) = j+1. For e ≥ 0, w(e) := min{w ≥ 0 : 2^w ≥ 5^e} = ⌈e log 5⌉, and w_{ab} := w(e_{ab}).

Definition 5.1 (the network N(G,r,β,order) on [g]).

Settings. A setting of node a is a pair (ℓ^{(a)}, y^{(a)}) with ℓ^{(a)} ∈ L^{δ(a)} and y^{(a)} ∈ {0,1}^{Q_a}. It is encoded as the concatenation of:

- for each b ∈ [g]\{a}, in increasing order, the *block* block_b(z_a), which is the w_{ab}-bit binary representation (lowest bit first) of Σ_{t=1}^{e_{ab}} κ(ℓ^{(a)}*{k_t})·5^{t−1}. Here k_1 < ⋯ < k*{e_{ab}} enumerate E(min(a,b), max(a,b)) in crossing order;
- the bits y^{(a)}_{cd}, for (c,d) ∈ Q_a in lexicographic order.

P_a := Σ_{b≠a} w_{ab} + |Q_a|. S_a is the set of strings of length P_a whose every block is < 5^{e_{ab}}. These are exactly the encodings, and the encoding is a bijection, so |S_a| = 5^{|δ(a)|} 2^{|Q_a|}.

Node functions. W_a(z_a) := σ_a(ℓ^{(a)}) · χ_a(ℓ^{(a)}, y^{(a)}) · I_a(ℓ^{(a)}).

Edges, for a < b. The edge function is

```
Ψ_{a,b}(z_a, z_b) := Π_{k∈E(a,b)} [ℓ^{(a)}_k = ℓ^{(b)}_k] · Π_{d=b+1}^{g} [y^{(a)}_{bd} =
P_{bd}(ℓ^{(b)})],
```

and Ψ_{b,a}(z_b, z_a) := Ψ_{a,b}(z_a, z_b). The number of shared bits is c_{a,b} = c_{b,a} := w_{ab} + (g − b). The factorization is

```
π_{ab}(z_a) := ( block_b(z_a), y^{(a)}_{b,b+1}, …, y^{(a)}_{b,g} ) ∈ {0,1}^{c_{a,b}},
π_{ba}(z_b) := ( block_a(z_b), P_{b,b+1}(ℓ^{(b)}), …, P_{b,g}(ℓ^{(b)}) ) ∈
{0,1}^{c_{a,b}},
f_{a,b}(z_a, θ) := [π_{ab}(z_a) = θ],     f_{b,a}(z_b, θ) := [π_{ba}(z_b) = θ].
```

What each bit means. The w_{ab} block bits on {a,b} are the base-5 code of the labels of the crossing edges between V_a and V_b. Both ends hold a copy, and Ψ tests that the copies are equal. For a < b, the g − b remaining bits on {a,b} are copies, held by a, of the parities P_{bd} for d > b. Node b computes these parities from its own labels on E(b,d).

Lemma 5.2. (f_{a,b}, f_{b,a}) factors Ψ_{a,b} through c_{a,b} bits.

Proof. Both ends list E(a,b) in the same order, and the base-5 encoding is injective. So block_b(z_a) = block_a(z_b) iff ℓ^{(a)} and ℓ^{(b)} agree on E(a,b). The remaining coordinates of π_{ab} and π_{ba} agree iff every copy is correct. Hence Ψ_{a,b} = [π_{ab}(z_a) = π_{ba}(z_b)], and Observation (b) after Definition 1 applies. ∎

Theorem 5.3. The value of N(G,r,β,order) is n·#HC(G), for every n ≥ 3, every β : V → [g], every root r and every crossing order.

Proof. A term Π_a W_a Π_{a<b} Ψ_{a,b} is nonzero only if all Ψ equal 1 (they are 0/1-valued). This means two things:

- i. There is ℓ ∈ L^m with ℓ^{(a)} = ℓ|δ(a) for every a. Each k lies in exactly δ(a_k) and δ(b_k), and Ψ_{a_k,b_k} tests agreement on k.
- ii. y^{(a)}*{cd} = P*{cd}(ℓ) for all a < c < d. Ψ_{a,c} tests this, and P_{cd}(ℓ^{(c)}) = P_{cd}(ℓ).

Conversely, every ℓ ∈ L^m determines exactly one z with (i) and (ii). So Z = Σ_ℓ Π_a σ_a(ℓ) χ_a(ℓ, P(ℓ)) I_a(ℓ) = n·#HC(G) by Theorem 4.6. ∎

Proposition 5.4 (parameters). Let μ_a := Σ_{v∈V_a} deg_G(v) and μ := max_a μ_a, and suppose min-degree(G) ≥ 2. Then N(G,r,β,order) is a network in the sense of Definition 2 with

```
P := w(μ) + g + g²,       B := 2μ + 1,       τ := 2^{1.5μ} · n^{C₂}    (C₂ absolute),
U = 2 Σ_{a<b} w_{ab} + 2·C(g,3)  ≤  2 log5 · m_× + g² + g³/3  ≤  2 log5 · |E| + g³,
```

where m_× is the number of crossing edges and τ is counted after a polynomial-time preparation. In particular P ≤ log5·μ + 2g² and gP ≤ log5·gμ + 2g³.

Proof.

Counting. Each crossing edge at a has exactly one end in V_a and each internal edge two, so 2|E_int(a)| + |δ(a)| = μ_a. Also w(e) ≤ e log5 + [e > 0] and w(x) ≥ x log5, so

```
Σ_{b≠a} w_{ab} ≤ |δ(a)|·log5 + (g−1) ≤ w(μ) + g − 1.
```

P_a ≤ P. P_a = Σ_b w_{ab} + C(g−a,2) ≤ w(μ) + g − 1 + g²/2 ≤ P.

(i). The copy bits on pairs at a number Σ_{b>a}(g−b) + Σ_{b<a}(g−a) = C(g−a,2) + (a−1)(g−a) ≤ g². So Σ_{b≠a} c_{a,b} ≤ w(μ) + g − 1 + g² ≤ P.

U. U = 2Σ_{a<b} c_{a,b}. Use Σ_{a<b}(g−b) = Σ_b (b−1)(g−b) = C(g,3) (choose the middle element of a triple) and #{a<b : e_{ab} > 0} ≤ g²/2. Finally g² + g³/3 ≤ g³ for g ≥ 2.

(ii). Ψ and f take values in {0,1}. |σ_a χ_a| = 1. In I_a, the product F_a(x)Γ_a(ℓ) has at most |E_int(a)| factors with ≤ 5 signed monomials each and |δ(a)| factors with ≤ 2 each, and there are at most 2^{|E_int(a)|} choices of x. So

```
|W_a| ≤ 10^{|E_int(a)|} 2^{|δ(a)|} ≤ 2^{(log 10/2)·2|E_int(a)| + |δ(a)|} ≤ 2^{1.67 μ_a}
< 2^B.
```

(iii). The preparation stores the graph, β, the crossing order, the lists E(a,b), w_{ab} and Q_a. Then:

- membership in S_a, decoding and encoding, π_{ab}, Ψ and f take poly(n + g) operations (base-5 arithmetic on ≤ P-bit numbers, with P ≤ poly(n));
- σ_a takes O(|δ(a)|²) and χ_a takes O(g³);
- I_a loops over ≤ 2^{|E_int(a)|} ≤ 2^{μ/2} vectors x. For each, it multiplies at most μ factors (≤ 5 monomials each) in Λ_a, represented densely by 2^{|Γ_a|} ≤ 4^{|V_a|} ≤ 2^{μ} integers of ≤ B = O(n) bits (|V_a| ≤ μ_a/2 because the minimum degree is ≥ 2), and reads one coefficient.

Altogether τ ≤ $2^{\{\mu/2\}\cdot 2\{\mu\}}$·poly(n). ∎

---

6. The algorithm and its running time

6.1 Preprocessing

Lemma 6.1 (degree reduction). Let $\Theta \geq 2$ and $H := \{v : \deg(v) > \Theta\}$. For each choice $\pi = (\pi_v)_{v\in H}$ of a 2-element subset π_v of the edges at v, let G_π be G minus every edge e at a vertex $v \in H$ with $e \notin \pi_v$. Then:

- $\#HC(G) = \Sigma_\pi \#HC(G_\pi)$;
- there are at most $\Pi_{v\in H} C(\deg v, 2) \leq n^{2|H|}$ branches, and $|H| < 2m/\Theta$;
- every G_π has vertex set V, at most m edges and maximum degree $\leq \Theta$.

Proof. Map a Hamiltonian cycle C of G to $\pi(C) := (E(C) \cap \delta(v))_{v\in H}$. C survives in $G\{\pi(C)\}$, since every edge of C at $v \in H$ lies in $\pi_v(C)$. Conversely, let C be a Hamiltonian cycle of G_π. For $v \in H$, $E(C) \cap \delta(v)$ is a 2-set contained in $\delta_{G_\pi}(v) \subseteq \pi_v$, so $\pi = \pi(C)$. Thus the pairs (π, C) with C a Hamiltonian cycle of G_π correspond bijectively to the Hamiltonian cycles of G. The remaining claims hold because $\Sigma \deg = 2m$ and vertices of H keep degree ≤ 2. ∎

Lemma 6.2 (degree-balanced partition). List V as $v_1,\dots,v_n$ in any order and let $S_t := \Sigma_{s<t} \deg(v_s)$. Put $\beta(v_t) := \min(g, \lfloor g\, S_t/(2m) \rfloor + 1)$. Then every blob has $\mu_a \leq 2m/g + \Delta$, where Δ is the maximum degree.

Proof. β is nondecreasing in t, so each blob is a segment $v_t,\dots,v_{t'}$.

- If $a < g$, the segment satisfies $(a-1)\cdot 2m/g \leq S_t \leq S_{t'} < a\cdot 2m/g$, so $\mu_a = S_{t'} - S_t + \deg(v_{t'}) < 2m/g + \Delta$.
- If $a = g$, then $\mu_g = 2m - S_t \leq 2m/g$. ∎

The algorithm, on input G.

0. If $n \leq 2$, or some vertex has degree < 2, output 0.
1. Choose ℓ (see §6.3 and §6.4). Put $K := K_\ell$ and $g := 3K^8$.
2. Apply Lemma 6.1 with $\Theta := \max(2, \lceil n/g \rceil)$. For each branch G_π, do steps 3–4. If G_π has a vertex of degree < 2 its count is 0, so skip it.
3. Take β from Lemma 6.2, any root r and any crossing order. Build $N(G_\pi, r, \beta, \text{order})$ implicitly: the preparation of Proposition 5.4, with P, B, P_a and $c_{a,b}$ computed exactly.
4. Run the procedure of Lemma 8(i) on it, with the algorithm of Theorem A(b) at level ℓ as the clique algorithm. Lemma 8 calls it on a (K, L)-clique with B' in the role of B. Divide the result by n.
5. Output the sum over the branches.

Correctness follows from Lemma 6.1, Theorem 5.3, Lemma 8 and Theorem A(b).

Bounds for every branch. Write m for $|E(G)|$; then $|E(G_\pi)| \leq m$ and min-degree$(G_\pi) \geq 2$. By Lemma 6.2, $\Delta \leq \Theta \leq n/g + 2$ and so $\mu \leq (2m + n)/g + 2$. Using $\log 5 \leq 2.33$ and $g \geq 3$, Proposition 5.4 gives:

$$gP \leq 2.33(2m + n) + 3g^3, \qquad U \leq 4.65\, m + g^3, \qquad B \leq 2(2m+n)/g + 5,$$
$$\log \tau \leq 1.5(2m+n)/g + 3 + C_2 \log n, \qquad L \leq gP + 1, \qquad B' \leq 2gP + 1 + g^2 B. \tag{6.1}$$

By (1.1) and (1.2), with $e := e_\ell \leq 1$ and $K^2 L \leq U + gP/K + K^2$, the time of steps 3–4 for one branch is 2^{X}, where

```
X ≤ 1 + max{ g log K,  e(U + gP/K + K²) + C₀K² log(K(L+B′)) + 2gP/K + log τ } + C₁
log(g(P+B)) + O(log n).

(6.2)
```

There are at most n^{2|H|} ≤ n^{4mg/n} branches, because |H| < 2m/Θ ≤ 2mg/n.

6.2 Imported hypotheses, checked

Lemma 8(i) needs a network on [g] with integer parameters P, B, τ ≥ 1, total U, and g = 3K⁸. Proposition 5.4 supplies these, with all values of all functions in Definition 2(ii) bounded and (iii) met with the stated τ.

Lemma 8 then needs an algorithm for every (K, L)-clique with |W| < 2^{B′}. Theorem A(b) at level ℓ, with λ = L and B = B′, gives T = $2^{\{e_\ell K^2 L\}(K(L+B'))\{C_0K^2\}}$. Its hypotheses are λ, B ≥ 1, which hold because P ≥ 1. Theorem A needs pieces: Theorem B, or Lemma 2 of [A]. Nothing in the clique algorithm depends on the graph, which reaches it only through the tables W̃*γ that Lemma 8 builds from W_a and f*{a,b}. “Balancing” is done by Lemma 8 itself: it searches the K^g groupings for one with ≤ 2g/K nodes per group and Y_{γ,γ′} ≤ L, which exists by the proof of Lemma 1 of [A].

6.3 Proof of Theorem 1

Let D ≥ 1, δ ∈ (0,1] and m ≤ Dn. Then 2m + n ≤ 3Dn. By Theorem A(a), e_ℓ → 0 and K_ℓ → ∞, so there is an ℓ, depending only on (δ, D), with e := e_ℓ ≤ δ/(40D) and K := K_ℓ ≥ 60D/δ. Fix it, and put g = 3K⁸, a constant. By (6.1), the terms of (6.2) are:

```
e·U          ≤ (δ/40D)(4.65 Dn + g³)          ≤ 0.12 δn + g³,
(e+2)·gP/K   ≤ 3(2.33·3Dn + 3g³)/K            ≤ 21 Dn/K + 9g³ ≤ 0.35 δn + 9g³,
log τ        ≤ 4.5 Dn/g + O(log n)            ≤ 0.075 δn + O(log n),
the rest     = O_{δ,D}(log n)        (g, K constant; L, B′, P, B = O_{δ,D}(n)).
```

So each branch costs 2^{0.55δn + O_{δ,D}(log n)}. There are at most n^{4Dg} = n^{O_{δ,D}(1)} branches, and the preparation is polynomial. The total is 2^{0.55δn + O_{δ,D}(log n)} = O(2^{δn}). ∎

Alternative route. Replace steps 1 and 4 by Lemma 1 + Proposition 3 of [A]: (K, e) achievable with e ≤ δ/(40D) and K ≥ 60D/δ. The exponent $C_1 2^{\{e(U+gP/K)+2gP/K\}(P+B)\{C_1\}\tau}$ is bounded by the same three lines.

6.4 Proof of Theorem 2

Use the pieces of Theorem B (ρ = 1/√2, α′ = ½, with c and x_0 absolute).

- *Small inputs.* For n < n_0 := K_0^{30}, where n_0 is an absolute constant, any exact method takes O(1) time.
- *Trivial cases.* Step 0 disposes of n ≤ 2 and of minimum degree < 2. Otherwise m ≥ n.
- *Choice of ℓ.* For n ≥ n_0, let ℓ be the largest index with K_ℓ^{30} ≤ n. K_ℓ is computable from the formula in Theorem A(a). Then K_{ℓ+1} > n^{1/30}, and K_{ℓ+1} ≤

K_ℓ^{10} gives n^{1/300} < K := K_ℓ ≤ n^{1/30}. So g = 3K⁸ ∈ [3n^{8/300}, 3n^{8/30}], and by Theorem A(a)

```
e = e_ℓ ≤ 0.85 ((3x₀+6)/log K)^{1/2} ≤ 0.85 (300(3x₀+6))^{1/2} / √(log n) =: c_e/
√(log n).
```

With 2m + n ≤ 3m, the terms of (6.2) and the branching are:

```
e·U                  ≤ c_e(4.65 m + 27 n^{0.8}) / √log n,
(e+2)·gP/K           ≤ 3(7m + 81 n^{0.8}) / n^{1/300}       = O(m · n^{−1/300}),
log τ                ≤ 4.5 m / g + O(log n)                 = O(m · n^{−8/300}),
e K² + C₀K² log(K(L+B′)) = O(n^{1/15} log n)               (L, B′ ≤ poly(n), as m ≤ n²),
g log K              = O(n^{8/30} log n),       C₁ log(g(P+B)) = O(log n),
log(#branches)       ≤ 4(m/n) g log n                       = O(m · n^{8/30−1} log n).
```

Every term except the first is O(m/√log n) uniformly, because m ≥ n and n^{0.8}, n^{1/15}log n and n^{8/30}log n are all o(n/√log n). Hence the total time is 2^{C*·m/√log n} for an absolute C*. ∎

Remarks.

- With the pieces of Lemma 2 of [A] only (ρ = 11/12), the same argument gives 2^{O(m/(log n)^{0.125})}.
- The leading term is e_ℓ·U ≈ e_ℓ·4.64|E|. Every other term is lower order, so any improvement of e_ℓ in Theorem A, for example better pieces (Track H), transfers verbatim.
- Like the bounds of [A], these are purely asymptotic. For Theorem 2 the bound beats 2^n only when e_ℓ·4.64m < n, which needs log n in the thousands even for m = 2n.

---

7. Corollaries

Corollary 7.1 (decision). G is Hamiltonian iff #HC(G) > 0. This takes one call of Theorem 1 or 2.

Corollary 7.2 (search, by self-reduction). Let H := G. For each edge e of G in turn: if #HC(H − e) > 0, set H := H − e. At the end, H is a Hamiltonian cycle.

Proof. The invariant is #HC(H) > 0. Let C be a Hamiltonian cycle of the final H. Every intermediate H contains the final H, hence C. Suppose some edge f of the final H were not in C. When f was examined, H − f contained C, so f would have been deleted. Hence the final H = C. There are m + 1 calls, each on a graph with n vertices and ≤ m edges. ∎

Corollary 7.3 (Hamiltonian paths). Let G⁺ be G plus a new vertex w adjacent to all of V. The undirected Hamiltonian paths of G correspond bijectively to the Hamiltonian cycles of G⁺, so #HP(G) = #HC(G⁺). G⁺ has n + 1 vertices and m + n ≤ (D+1)(n+1) edges. The apex w has degree n and is handled by Lemma 6.1. Search works as in Corollary 7.2. For s–t paths, join one new vertex to s and t only. ∎

---

8. Numerical verification (exact integers)

Scripts. `research/scripts/proof_lib.py` implements §§2–5 literally and independently of `lowrank_grassmann.py`:

- an exterior algebra over Z with a *randomized* generator order;
- the pieces table, ε, σ_a, P_{ab} and χ_a;
- the network of Definition 5.1, with fixed-length bit-string settings, base-5 blocks, copy bits, W_a, Ψ_{a,b}, c_{a,b} and f_{a,b} via the projections π_{ab};
- two network evaluators that use W, Ψ and f only as black boxes. One is brute force over Π_a S_a; it is exact and stops extending a partial product only when it is already 0. The other is the Lemma 1/Lemma 8 grouping into K = 2 or 3 groups, which builds the tables W̃_γ and sums the clique.

Runs.

- `python3 proof_checks.py 11 60`: 60 instances, n ≤ 12, all identities. Prints ALL CHECKS PASSED, in about 2 minutes. Smaller runs with seeds 1, 3, 5 and 7 also pass.
- `python3 proof_params.py 7`: prints ALL CHECKS PASSED.

Instances. There are five kinds:

- random graphs (n = 4..9) with *arbitrary* random maps β, empty blobs allowed;
- planted blob graphs with random blob relabelling;
- planted Hamiltonian cycles;
- g ∈ {4,5} instances whose planted Hamiltonian cycle visits the blobs in an interleaving cyclic order;
- g ∈ {4,5} instances with a planted *disconnected* 2-factor, whose non-root cycle visits the blobs in interleaving order (0,2,1,3,...).

Every instance uses a random root, a random crossing order and a random generator order.

Checks. The number in parentheses is the count of individual checks in the run with seed 11.

check	identity	count
C1	n#HC = Σ_X τ(V,X) and τ ∈ {0,n}, with τ by brute-force spanning-tree enumeration (Lemma 3.1)	60 / 135
C2	τ = det L_X^{(r)} (Bareiss) = [Ω_V]Π(1+q_e) (Lemma 3.3)	135
C3	Lemma 3.2 on random integer matrices	200
C4	Koszul rule on random homogeneous sequences and permutations (Lemma 2.2)	300
C5	block factorization of [Ω] on random elements (Lemma 2.3)	180
C6	Lemma 4.1 (a), (b), for both orientations and with root ends	1304 / 5216

check	identity	count
C7	Proposition 4.2 as an *equality of algebra elements*, over all labellings when 5^m ≤ 4000, else 4000 random ones	162 875
C8	Proposition 4.4, and Π χ_a = interleaving sign (Lemma 4.5)	162 875 each
C9	Proposition 4.2 with fresh generators, so that nothing cancels, for random g ≤ 7, ≤ 8 chords, random parities and orders; Lemma 4.3	400 / 1033
C10	Definition 2 for the network: P_a ≤ P, (i), symmetry, U = 2Σw_{ab} + 2C(g,3), w_{ab} = ⌈e log5⌉, |S_a| = $5^{\{\|\delta(a)\|\}2}$, |Q_a|, |W_a| ≤ $10^{\{\|E_int\|\}2}$, |δ| < 2^B, and Ψ = Σ_θ f f′ (on all θ ∈ {0,1}^{c} when small)	≈ 594 000
C11	Σ_ℓ ε Π I_a = n#HC (Theorem 4.6); value of the network by brute force = n#HC (Theorem 5.3); value via the K-grouping = n#HC	60 / 60 / 103
C12	ablations: without the interleaving sign 20/60 instances are wrong, without the local signs 8/60, without any sign 18/60	—

C12 shows that the interleaving sign is needed. It matters exactly when odd labels sit on interleaving chords, which in practice happens on disconnected 2-factors whose non-root cycle winds through the blobs in interleaving order. There the odd terms are what cancel τ to 0.

`proof_params.py` checks the following:

check	identity
P1	Lemma 6.1 (#HC(G) = Σ_π #HC(G_π), maximum degree and edge count of the branches), 109 random graphs, thresholds 2–5
P2	Lemma 6.2 in the exact form g·μ_a ≤ 2m + gΔ, 400 graphs, g ≤ 12
P3	Σ_b(b−1)(g−b) = C(g,3), |Q_a| = C(g−a,2), copy bits at a ≤ $3g^2/4$, g^2 + 2C(g,3) ≤ g^3, Σ_b w(e_b) ≤ w(Σe_b) + #b

check	identity
P4	the self-reduction of Corollary 7.2 returns exactly a Hamiltonian cycle (34 graphs)
P5	$\|H\|\cdot\Theta < 2m$

9. What is proved, what is assumed, what is conjectural

Proved here (paper level, complete proofs above):

- Lemma 3.1, Lemma 3.2 and Lemma 3.3 (matrix-tree theorem via Grassmann/Cauchy–Binet, with no Gaussian-integral conventions needed), and Corollary 3.4;
- the Grassmann facts 2.1–2.3, proved;
- the pieces (Lemma 4.1), the sign rule for arbitrary partitions and crossing orders (Proposition 4.2), and its factorization into blob-local signs and the interleaving form (Lemma 4.3, Proposition 4.4, Lemma 4.5);
- the blob factorization (Theorem 4.6);
- the network, its factorization, its value (Lemma 5.2, Theorem 5.3) and all its parameters (Proposition 5.4);
- the reductions (Lemmas 6.1 and 6.2) and the corollaries;
- the parameter bookkeeping (6.1)–(6.2) and the derivations of Theorems 1 and 2 *from* the imported statements.

Assumed (imported from [A]; we use nothing else of [A]):

1. **Lemma 8, case (i)**, in the form (1.1). In Lean this is `ExplicitCost.groupingExplicit` (`proof/rates/ExplicitCost/GroupingLemma.lean`). Lean's model differs: operations are polynomial-time steps or queries, there are memory bounds, and K, g, P, B are given in unary. Our network fits the query model: W_a, Ψ and f are oracles of cost τ.
2. **Theorem A(a),(b)**. (b) is `ExplicitCost.recursionSolves` in Lean (`PaperIndex.cliquesOfAllLevelsInCountedOperations`). (a) is elementary arithmetic on (40) of [A].
3. **Pieces.** Theorem B (`PiecesCounted.halfPieces` in Lean) for Theorem 2. Lemma 2 of [A] or Theorem B for Theorem 1.
4. For the alternative route to Theorem 1: Lemma 1 (`Printed.groupingLemma`, with Lean's memory caveat) and Proposition 3 of [A].

Composing items 1 and 2 with *our* network is done here on paper only. [A]'s Lean instantiates the chain only for CNF networks. The cost model is that of [A] (operations on words). Our big-integer arithmetic on O(n)-bit numbers costs poly(n) operations and is absorbed in τ and in the polynomial factors. If [A]'s Theorem A(b) or Lemma 8 failed, Theorems 1 and 2 would lose their proofs, while §§2–5 would be unaffected. Uniformity in Theorem 2 uses the constant c of Theorem B (through x_0). If c is not explicit, the statement still asserts the existence of one algorithm, as in [A].

Not claimed / conjectural:

- *Practicality.* The crossover point is astronomically large, as in [A].
- *Extensions* (`exact-extensions.md`): directed HC (directed matrix-tree theorem, 4 labels per arc), weighted/TSP counting by polynomial weights, and other problems. These are plausible by the same template, but are not proved here.

- *Better exponents.* Whether e_ℓ, and hence the exponent, can be improved below 1/√log n is open. It is tied to better pieces (ρ → ½ would give 2^{O(m/log m)}).
- *Dense graphs.* Theorem 2 beats 2^n only for m = o(n√log n). Nothing is claimed for dense graphs beyond the classical O*(2^n).

Gaps. None found in the mathematics of §§2–6. The points that needed care are listed at the end of §0. No independent referee report exists yet; see the addendum below.

Addendum: referee report

No referee report was produced. The first referee session was lost in a container restart before it wrote `review-exact-count.md`, and a new referee is being started. This write-up has therefore **not yet been independently refereed**. Any issues the new report raises should be answered here. A repeat run after the restart (`proof_checks.py 3 12`, `proof_params.py 7`) still prints ALL CHECKS PASSED.

Appendix: independent referee report

Referee, 2026-10-27. Status: COMPLETE. Verdict: **SOUND, conditional on Anthropic's [A]** (Theorem A(b) / Lemma 8(i), equivalently Lemma 1 with e* = 0). No errors found in steps (1)-(4). Two points that looked like gaps in the one-paragraph claim are already handled correctly in research/exact-hc-theorem.md (details below).

Method

My own derivation and code came first; I read research/exact-hc-theorem.md only after that. Scripts (exact integers, all written from scratch): - scripts/review_exact_count.py: brute-force #HC, 2-factor enumeration, Bareiss det, a global Grassmann/Berezin integrator, a blob evaluator built from my own sign derivation, and a literal network (Definition 2 of [A]) with copy bits summed over {0,1} with their Psi indicators (g <= 5). Two crossing-edge alphabets: 6 states, and a 5-state product trick (1+loc_i)(1+loc_j) - loc_i loc_j. Half the blob tests carry random integer weights w_e on "edge used" and u_e in {-3..3}{0} on the bilinear terms. Target: sum_X prod w_e det L_r(X;u). - scripts/review_exact_count_ablate.py: drops the global interleaving sign (unweighted and weighted). - scripts/review_exact_count_interleave.py: graphs built so that crossing edges pile up on interleaving bundles.

(a) Berezin = det

About 500 (graph, arbitrary subgraph X, every root) checks, plus about 115 checks of the sum over all 2-factors. All equal. Integral = coefficient of prod_{v != r} psibar_v psi_v.

(b) Sign rule (own derivation) and blob evaluation

Each bilinear term is a pair of generators. Pairs are even, so they commute. Put each pair in blob order (the generator in the lower blob first). The sign is then (-1)^{number of interleaving pairs of pairs}. - Pairs that meet in exactly one blob are decided by that blob alone. - Two pairs in the same bundle ab interleave iff 1 + [x_p<x_q] + [y_p<y_q] is odd. This splits into C(N_ab,2) + (a-local) + (b-local), so it is local too. - Pairs on disjoint chords give exactly sum_{a<c<b<d} N_ab N_cd.

This matches claim (3). Observation: every blob has an even number of odd generators, so (P_ab) is a cycle-space vector of K_g. Because of that the global sign is nontrivial only on a few % of random instances.

Tests: n = 3..10, including Petersen, prisms, K4, K5, K33, K34 and rings of triangles, plus random and cubic-like graphs. g = 2..6, with random, very unequal and empty blobs, a random root and a random local order. About 790 blob evaluations (638 in the main suite, both alphabets) and 297 literal-network evaluations. **0 failures.** Without the global sign, 8 instances give the wrong answer, so the tests do exercise that term.

One failure appeared during development. It came from misplaced weights in my own harness, not from the claim, and was fixed.

The claim's "5 choices (unused, or one of 4 bilinear terms)" is literally one state short: "used with term 1" is missing. The write-up's labels {N,0,1,2,3}, with label 0 = 1* (1+psibar_t psi_t), merge two terms correctly. I did not test those exact labels. My 5-state alphabet is a different merge and passes, and the difference only changes a constant.

(c) Fit to [A]

- Settings are the base-5 codes of the incident crossing labels plus copy bits (subsets of {0,1}^{P_a} are allowed). The node table W_a is a signed integer table that includes the (-1)^{P_ab * copy} factors.
- Psi is a genuine equality test (Def 1(b)).
- Totals: U <= 2 log5 m + g^3 and B = O(n/g). log tau <= 1.5 mu, with mu <= (2m+n)/g + 2.
- Lemma 8(i) with Theorem A(b) gives an exponent of about e*4.65m + 3*gP/K + log tau + O(g^3) + O(K^2 log n). This is <= delta n for e <= delta/40D, K >= 60D/delta and g = 3K^8.
- Nothing hidden blows up. The C(g,4) interleavings cost only O(g^3) copy bits, a constant for fixed delta. The factors K^g and (K(L+B'))^{C0 K^2} are polynomial.
- Unbounded degree is the one real trap: a blob holding a degree-Theta(n) vertex makes P = Theta(n) and 2gP/K = Theta(K^7 n). The write-up fixes this (Lemma 6.1) by branching on the two cycle edges at each vertex of degree > n/g (at most n^{4Dg} branches, polynomial for fixed g), followed by a degree-balanced partition. Correct.

(d) Plausibility / literature

- Identity (1) is elementary folklore: a 2-factor has n spanning trees if it is a Hamiltonian cycle and 0 if it is disconnected. Pairing cycle covers or subgraphs with Kirchhoff counts to enforce connectivity is standard (e.g. Bjorklund's determinant sums).
- The fermionic matrix-tree form is Caracciolo-Jacobsen-Saleur-Sokal-Sportiello (2004).
- The result contradicts (#)ETH for sparse #HC, and hence SETH-type assumptions, all of which [A] already claims to refute. I know of no other robust conditional or unconditional bound that it breaks. Pathwidth lower bounds of the (6-eps)^pw kind concern graphs where N >> pw.

What remains conditional

Everything algorithmic rests on [A]: Theorem A(b), Lemma 8 and the existence of pieces. Steps (1)-(4) and the bookkeeping are sound. I did not build or re-verify the Lean files.